Habit Builder: Project Flow

1. IDEA

Gamified habit tracker that builds daily routines through streaks, reminders, heatmaps, and weekly analytics.

2. RESEARCH

- **Competitors:** Habitica, Streaks, Loop, Way of Life
- **Gaps:** weak streak-recovery logic, cluttered UI, unreliable reminders
- **Feasibility:** Web Notifications API, Service Workers, Web Push

3. DEFINE THE USER

- **Persona:** student or working adult (18–35), busy, wants simple daily consistency
- **Pain points:** forgets habits, loses motivation after breaking a streak
- **Needs:** quick check-off, timely reminders, visible progress

4. PRD

Must have

- Auth (register/login)
- Habit CRUD (name, frequency, target time)
- Daily checklist
- Streak counter (current and longest)
- Heatmap (calendar view)
- Reminders (local/push)
- Weekly analytics

Nice to have: badges/XP, dark mode

Out of scope: social features, payments

Success metrics: habit completed in ≤2 taps, reminder delivery works, streak accuracy is 100%

5. SYSTEM DESIGN

- **Frontend:** React + Tailwind + Recharts, PWA
- **Backend:** Node.js/Express (REST API)
- **DB:** PostgreSQL or MongoDB
- **Scheduler:** node-cron + Web Push
- **Tables:** users, habits, habit_logs (habit_id, date, done), reminders
- **Streak logic:** count consecutive scheduled days with a log, reset on a missed day, and store the longest streak
- **Flow:** Client → REST API → DB; Scheduler → Push service → Client

6. PROJECT RULES

- Git with feature branches and clear commit messages
- One task = one branch = one PR (self-review)
- Store dates in UTC, display in the user's timezone
- Validate all inputs; hash passwords; keep secrets in .env
- Write unit tests for streak logic
- Use a consistent code style (ESLint and Prettier)

7. TASK BREAKDOWN

Tasks

- Repo, project structure, DB, auth
- Habit CRUD, daily checklist API and UI
- Streak engine and tests
- Heatmap, progress UI
- Local notifications, push, scheduler
- Weekly stats and charts
- Responsive UI, error handling

8. DEVELOP

Build in this order: auth → habits → checklist → streaks → heatmap → reminders → analytics. Test each feature before starting the next.

9. DEPLOY

- **Frontend:** Vercel or Netlify
- **Backend:** Render or Railway
- **DB:** managed instance (Supabase, Neon, or Atlas)
- **Extras:** CI/CD via GitHub Actions, HTTPS (required for push)

10. REVIEW

- Test with 3–5 real users
- Check streak edge cases (timezones, missed days)
- Verify reminder reliability
- Collect feedback

11. ITERATE

Fix bugs, then prioritize feedback and add nice-to-have features (badges, XP, dark mode). Release updates in small versions.

12. TIMELINE & ESTIMATION

- **Phase 1: Setup & Auth (Days 1–3)** — Repository setup, DB schema, and Authentication (Register/Login).
- **Phase 2: Core Features (Days 4–8)** — Habit CRUD and Daily Checklist API + UI.
- **Phase 3: Streaks & Heatmap (Days 9–13)** — Streak engine development, unit testing for streak logic, Heatmap calendar view, and Progress UI.
- **Phase 4: Reminders & Analytics (Days 14–18)** — node-cron scheduler setup, Web Push notifications, and Weekly analytics/charts.
- **Phase 5: Polish & Responsiveness (Days 19–21)** — Mobile responsiveness, error handling, and integration testing.
- **Phase 6: Deployment & CI/CD (Days 22–24)** — Frontend deployment (Vercel/Netlify), Backend hosting (Render/Railway), Managed DB setup, and GitHub Actions CI/CD.
- **Phase 7: Testing & Review (Days 25–27)** — Testing with 3–5 real users, timezone edge cases, and reminder delivery checks.

- **Phase 8: Buffer & Iteration (Days 28–30)** — Bug fixes, priority user feedback updates, and nice-to-have features (Dark Mode, Badges, XP).